3 Handle retry conditions (optional). Configure the `shouldRetryThisError` option for retrying requests when specific errors occur. You can customize the behavior by specifying which errors should trigger a retry:

```
const retryableStatusCodes = [429, 500];
const supportedModel = createFallback({
  models: [
...
  ],
  shouldRetryThisError: (error) => {
    retryableStatusCodes.includes(error.statusCode)
  },
});
```

The code specifies a list of retryable status codes, such as 429 and 500 (Internal Server Error), which indicate conditions under which requests should be retried. Of course, you are free to modify the logic based on a specific error code or response type.

Reviewing the ai-fallback implementation

While you are setting up this feature, it's also important to review the existing code of the library's main class to see how it works behind the scenes. The code defines a `FallbackModel` class that implements the `LanguageModelV1` interface, allowing for seamless switching between multiple AI models based on their performance and availability:

```
export class FallbackModel implements LanguageModelV1 {
   readonly specificationVersion = 'v1'
   // rest of code here
}
```

The `LanguageModelV1` interface is essentially an extension to the language specification model that I explained in chapter 3. This examination will provide valuable insights into how to implement similar extensions effectively in the future.

Retry logic, fallback, and token limit mechanisms are all effective solutions for addressing problems that arise when an AI provider becomes unavailable or stops functioning. These strategies help ensure that customers receive uninterrupted service by providing alternative options when primary models fail. However, relying on these mechanisms is only one part of the equation; you still need to actively monitor the health and reliability of the AI services being utilized. Usually this involves using an external service provider.

We won't go into detail about which providers to use for logging and monitoring web applications, but there are two popular options:

- *Sentry*—A free tier is available for error tracking and performance monitoring. Sentry helps developers identify and fix problems in real time by capturing exceptions and performance data.